

01 OVERVIEW

SiteWeather is a React single-page dashboard that turns raw OpenWeather data into go / no-go decisions for outdoor construction crews. It presents live site conditions, a five-day forecast, a threshold-driven alert engine, derived heat- and cold-stress meteorology, and a static safety reference for the supervisor deciding whether work can safely proceed. Built as university coursework (GUI module, Group 8) and released under MIT.

02 APPLICATION SURFACE — SEVEN VIEWS

- Dashboard** live conditions over a condition-matched animated sky background, metric tiles, five-day strip and auto-generated safety recommendations.
- Forecast** five-day outlook with drag-scrollable three-hourly strips per day.
- Report** derived meteorology — dew point, wet bulb, heat index, wind chill, absolute humidity, vapour-pressure deficit, Beaufort level, wind compass, pressure trend, visibility and UV bands.
- Locations** search over ~120 world cities plus twelve UK presets, favourites persisted to localStorage, a GPS card and set-as-default-city.
- Alerts** rule-engine output rendered as severity-coded cards, with a One Call 3.0 fallback fetch for UV.
- Safety** ten-question FAQ accordion and six external OSHA / industry references.
- Settings** dark mode, °C / °F, km/h / mph and browser-notification opt-in.

03 ARCHITECTURE & DATA FLOW

App.jsx is the single stateful container. It holds activeTab and dispatches through a switch-based router — there is no react-router and no state library. Weather is fetched once per change of defaultCity and passed down as props, so every view renders from one shared snapshot. services/openWeather.js wraps the OpenWeather 2.5 /weather and /forecast endpoints, keyed by VITE_OPENWEATHER_API_KEY and always requested in metric; unit conversion is presentational only (utils/units.js). buildFiveDayForecast() groups three-hourly entries by ISO date, elects the 12:00 sample as each day's representative and attaches the full hourly series beneath it. A 401 is surfaced as a specific "key invalid or not yet activated" message. Persistence is localStorage through usePreferences() (default city, units, alert opt-in); theme lives in a React context that toggles Tailwind's dark class on the document root. useGeolocation() requests coordinates once with an eight-second timeout, listens for permission changes via the Permissions API, and degrades to the default city with an explanatory message rather than failing.

04 ALERT & SAFETY RULE ENGINE

alertRules is a declarative table; generateAlerts() walks each parameter and takes the first rule whose min / max bounds are satisfied, so the most severe band wins.

PARAMETER	MEDIUM / ADVISORY	HIGH / WARNING
Wind gust	25 – 34 km/h	35 km/h and above
Rainfall (1h / 3h)	5 – 9 mm	10 mm and above
Temperature — heat	20 – 29 °C	30 °C and above
Temperature — cold	1 – 5 °C	0 °C and below
Heat index (Rothfusz)	25 – 34 °C	35 °C and above
UV index	3 – 5	6 and above
Wet bulb (Report)	25 °C caution	28 danger, 32 stop work

App.jsx re-derives a single top alert for the persistent banner shown above every page and fires a browser Notification for it, de-duplicated on title plus body. The header's project status is a coarser read of the same inputs: Weather Risk from 25 km/h or 10 mm, Monitor Conditions from 15 km/h or 5 mm, otherwise On Schedule. Dashboard recommendations are generated from wind, rain probability and temperature, defaulting to "conditions are favourable for work".

05 INTERFACE & PRESENTATION

Layout pairs a floating island bottom-nav on mobile with a desktop sidebar carrying a live site clock. The header adds location, project status, a search across all seven views and a bell opening a notification modal built from the same alert rules. Dashboard backgrounds are animated sky GIFs picked from the OpenWeather condition id, split day / night on sunrise and sunset. Public Sans and Material Symbols throughout; dark mode is class-based and persisted.

11 OBSERVATIONS & KNOWN GAPS

- getWeatherBackground() is duplicated in Dashboard.jsx and openWeather.js, and the Dashboard copy maps thunderstorms to sky-storm.gif, which does not exist — the shipped asset is sky-thunder.gif.
- heatIndex() and generateAlerts() are re-implemented in NotificationModal.jsx rather than imported from Alerts.jsx, so thresholds must be changed in two places.
- AdminPanel.jsx and WorkSafetyCard.jsx are complete but mounted nowhere; components/LocationCard.jsx is shadowed by a local definition inside Locations.jsx.
- Geolocation is used on the Locations page only — the primary fetch stays city-based, so the dashboard does not follow the user's actual position.
- The API key is a VITE_ variable and is exposed in the client bundle; .env sits in the working tree and must stay untracked. There are no automated tests.

06 STACK

React 19.2 · Vite 7.3 · Tailwind CSS 3.4 (class-based dark mode) · PostCSS with Autoprefixer · ESLint 9 flat config, react-hooks and react-refresh · OpenWeather 2.5 plus One Call 3.0 for UV. Only two runtime dependencies: react and react-dom.

07 AT A GLANCE

Source files	27
Lines of code	3,189
Pages / components	7 / 10
Hooks / contexts	2 / 1
Commits	74
Contributors	6
Remote branches	11
Test suites	0
Licence	MIT

08 REPOSITORY LAYOUT

src/pages/	Dashboard, Forecast, WeatherReport, Locations, Alerts, Safety, Settings
src/components/	Header, Navigation, Layout, SearchBar, NotificationModal, AlertBanner, MetricTile, ForecastItem, LocationCard, WorkSafetyCard, AdminPanel
src/services/	openWeather.js — API wrapper, forecast shaping, background selection
src/hooks/	useGeolocation, usePreference
src/context/	ThemeContext
src/utils/	units.js — temperature and speed conversion
public/assets/	seven animated sky GIFs and site patterns
guides/	git-guide.txt, project-guide.txt

09 RUNNING IT

- npm install
 - .env — VITE_OPENWEATHER_API_KEY=your key
 - npm run dev
- Scripts: dev, build, lint, preview. Allow location access for GPS weather; if blocked the app falls back to the saved default city (London).

10 TEAM & PROCESS

Ashar Adeel	31 commits
Aayan Aboobakar	13
Ioan-Adrian Fanea	12
Arjun Sambee	10
Prabhdeep Mall	9

guides/ records the agreed conventions: commit prefixes (feat, fix, docs, refactor, chore, style, test) and branch naming (feat/, fix/, chore/, docs/) always cut from an updated main, which is kept stable and demo-ready. Work reaches main through pull requests.